

Data Structures with C

Lecture 4: Data Structure Operations, Algorithm Analysis and Time-Space Trade-off

Dr. Siddhant Thapliyal

August 1, 2026

Data Structure Operations

The common operations performed on data structures are:

- ➊ **Traversal:** Accessing every element exactly once.
- ➋ **Insertion:** Adding a new element.
- ➌ **Deletion:** Removing an existing element.
- ➍ **Searching:** Finding the location of an element.
- ➎ **Sorting:** Arranging elements in a specific order.
- ➏ **Merging:** Combining two similar data structures.
- ➐ **Updating:** Modifying an existing element.

What is Algorithm Analysis?

Definition

Algorithm analysis determines the amount of resources required by an algorithm during its execution.

The two major resources are:

- **Execution time**
- **Computer memory**

The efficiency of an algorithm is generally expressed as a function of the input size n .

Why Analyse Algorithms?

Algorithm analysis helps us to:

- Compare different algorithms for the same problem.
- Predict performance for large input sizes.
- Select an efficient data structure.
- Reduce execution time.
- Reduce memory consumption.
- Determine the scalability of a program.

Time Complexity

Definition

Time complexity is the running time of an algorithm expressed as a function of its input size.

$T(n)$ = Number of elementary operations performed

Examples:

- Accessing one array element: $O(1)$
- Traversing an array: $O(n)$
- Binary search: $O(\log n)$
- Comparing every pair: $O(n^2)$

Definition

Space complexity is the total amount of memory required by an algorithm during its execution as a function of input size.

$$S(n) = \text{Fixed Space} + \text{Variable Space}$$

- **Fixed part:** Instructions, constants, simple variables and fixed-size arrays.
- **Variable part:** Dynamic memory, recursion stack and variable-size structures.

Fixed and Variable Space

Fixed Space

- Program instructions
- Constants
- Simple variables
- Fixed-size arrays
- Static structures

Variable Space

- Dynamic arrays
- Linked structures
- Recursion stack
- Dynamically allocated memory
- Input-dependent storage

Cases of Time Complexity

Case	Meaning
Best Case	Minimum running time for an input of size n .
Average Case	Expected running time for randomly distributed inputs.
Worst Case	Maximum running time for any input of size n .
Amortized Case	Average cost of a sequence of related operations.

Best-Case Analysis

The best case represents the most favourable input.

Linear Search

If the required element is at the first position:

$$T_{\text{best}}(n) = 1$$

Therefore,

$$T_{\text{best}}(n) = O(1)$$

Best-case analysis alone is generally not sufficient for selecting an algorithm.

Worst-Case Analysis

The worst case represents the maximum number of operations performed.

Linear Search

If the element is at the last position or is not present:

$$T_{\text{worst}}(n) = n$$

Therefore,

$$T_{\text{worst}}(n) = O(n)$$

Worst-case analysis provides an upper bound on running time.

Average-Case Analysis

Average-case complexity estimates the expected running time.

For linear search, assuming that the element is equally likely to occur at any position:

$$T_{\text{average}}(n) = \frac{1 + 2 + 3 + \dots + n}{n}$$

$$T_{\text{average}}(n) = \frac{n(n+1)}{2n} = \frac{n+1}{2}$$

Therefore,

$$T_{\text{average}}(n) = O(n)$$

Amortized Analysis

Definition

Amortized analysis determines the average cost per operation over a sequence of operations.

Dynamic Array

Most insertions take constant time.

Occasionally, the array must be resized and all elements must be copied.

Although one resizing operation may take $O(n)$ time, the amortized cost of each insertion remains:

$$O(1)$$

Time-Space Trade-off

Concept

An algorithm may use additional memory to reduce execution time, or use less memory at the cost of additional execution time.

- More memory may provide faster execution.
- Less memory may increase computation time.
- The choice depends on system requirements.

Time Efficiency $\longleftrightarrow$ Space Efficiency

Examples of Time–Space Trade-off

Problem	Less Space	Less Time
Searching	Linear search	Hash table
Repeated calculations	Recalculate values	Store results using memoization
File access	Sequential access	Indexed access
Recursion	Iterative solution	Recursive solution with call stack

Expressing Complexity

The complexity of an algorithm is represented using a function:

$$f(n)$$

where n is the size of the input.

Common complexity classes are:

$$O(1), \quad O(\log n), \quad O(n), \quad O(n \log n), \quad O(n^2), \quad O(2^n), \quad O(n!)$$

Big-O notation is generally used to express the upper bound of complexity.

Order of Growth

$$O(1) < O(\log n) < O(n) < O(n \log n) < O(n^2) < O(n^3) < O(2^n) < O(n!)$$

Observation

As input size increases, algorithms with lower growth rates perform better.

For large values of n , constants and lower-order terms are ignored.

$$3n^2 + 5n + 10 = O(n^2)$$

Constant-Time Algorithm

```
int firstElement(int arr[])  
{  
    return arr[0];  
}
```

The statement executes only once, regardless of input size.

$$T(n) = 1$$

Therefore,

$$T(n) = O(1)$$

Linear Loop

```
for (i = 0; i < n; i++)  
{  
    printf("%d", i);  
}
```

The loop executes n times.

$$T(n) = n$$

Therefore,

$$T(n) = O(n)$$

Linear Loop with Step Size

```
for (i = 0; i < n; i += 2)
{
    printf("%d", i);
}
```

The loop executes approximately $\frac{n}{2}$ times.

$$T(n) = \frac{n}{2}$$

Ignoring the constant factor:

$$T(n) = O(n)$$

Logarithmic Loop

```
for (i = 1; i < n; i *= 2)
{
    printf("%d", i);
}
```

Values of i are:

$$1, 2, 4, 8, 16, \dots, n$$

After k iterations:

$$2^k = n$$

$$k = \log_2 n$$

Therefore,

$$T(n) = O(\log n)$$

Another Logarithmic Loop

```
for (i = n; i >= 1; i /= 2)
{
    printf("%d", i);
}
```

The value of i is divided by 2 in every iteration.

$$n, \frac{n}{2}, \frac{n}{4}, \frac{n}{8}, \dots, 1$$

Therefore,

$$T(n) = O(\log n)$$

Linear-Logarithmic Loop

```
for (i = 0; i < n; i++)  
{  
    for (j = 1; j < n; j *= 2)  
    {  
        printf("*");  
    }  
}
```

Outer loop:

n

Inner loop:

$\log n$

Total number of operations:

Quadratic Loop

```
for (i = 0; i < n; i++)  
{  
    for (j = 0; j < n; j++)  
    {  
        printf("*");  
    }  
}
```

Outer loop executes n times.

Inner loop executes n times for every outer-loop iteration.

$$T(n) = n \times n = n^2$$

Therefore,

$$T(n) = O(n^2)$$

Dependent Quadratic Loop

```
for (i = 0; i < n; i++)  
{  
    for (j = 0; j <= i; j++)  
    {  
        printf("*");  
    }  
}
```

The total number of iterations is:

$$1 + 2 + 3 + \cdots + n$$

$$T(n) = \frac{n(n+1)}{2}$$

$$T(n) = \frac{n^2 + n}{2}$$

Sequential Loops

```
for (i = 0; i < n; i++)  
    printf("%d", i);  
  
for (j = 0; j < n; j++)  
    printf("%d", j);
```

The loops are sequential, not nested.

$$T(n) = n + n = 2n$$

Ignoring the constant:

$$T(n) = O(n)$$

Rules for Analysing Loops

- 1 Constant statements take $O(1)$ time.
- 2 Sequential operations are added.
- 3 Nested-loop complexities are multiplied.
- 4 Ignore constant factors.
- 5 Ignore lower-order terms.
- 6 Consider the dominant operation.

Example:

$$T(n) = 4n^2 + 3n + 20$$

The dominant term is n^2 .

$$T(n) = O(n^2)$$

Common Complexity Examples

Operation/Algorithm	Complexity
Array element access	$O(1)$
Linear search	$O(n)$
Binary search	$O(\log n)$
Merge sort	$O(n \log n)$
Bubble sort	$O(n^2)$
Binary-tree traversal	$O(n)$
Hash-table search, average case	$O(1)$

Worked Example

Find the complexity of:

$$T(n) = 5n^2 + 4n + 7$$

- 1 Identify the highest-order term: n^2
- 2 Ignore the coefficient 5.
- 3 Ignore lower-order terms $4n + 7$.

Therefore,

$$T(n) = O(n^2)$$

Practice Questions

- 1 Differentiate between time and space complexity.
- 2 Explain fixed and variable memory requirements.
- 3 Differentiate between best-case and worst-case complexity.
- 4 What is amortized analysis?
- 5 Explain the time–space trade-off with an example.
- 6 Find the complexity of a loop in which i doubles after each iteration.
- 7 Find the complexity of two nested loops, each executing n times.

Class Activity

Determine the time complexity of the following code:

```
for (i = 1; i <= n; i++)  
{  
    for (j = 1; j <= i; j++)  
    {  
        printf("%d", j);  
    }  
}
```

Class Activity

Determine the time complexity of the following code:

```
for (i = 1; i <= n; i++)  
{  
    for (j = 1; j <= i; j++)  
    {  
        printf("%d", j);  
    }  
}
```

$$1 + 2 + 3 + \dots + n = \frac{n(n+1)}{2}$$

$$O(n^2)$$

Questions?

Thank You